



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN GENERAL DE PREGRADO
INGENIERÍA INFORMÁTICA
ASIGNATURA: INGENIERÍA DEL SOFTWARE II
SECCIÓN 1

“Ejercicio 2: La Falacia de los Recursos y el Cono de Incertidumbre”

Docente:

Félix Márquez

Alumnos:

Luis Millan V - 30.040.213

Endrys Flores V - 30.451.556

Robert Castro V - 30.994.049

Jesus Baez V - 26.753.871

Ciudad Guayana, septiembre del 2026.

Contexto del caso

Faltan 2 meses para la fecha límite de un proyecto de software crítico, que está retrasado. El Director de la Empresa ordena duplicar el equipo de desarrollo (de 5 a 10 programadores) para recuperar el tiempo perdido y cumplir con el cronograma de Gantt original, ignorando la Ley de Brooks.

Por qué falla la ecuación $\text{Tiempo} = \text{Esfuerzo} / \text{Recursos}$

La ecuación $\text{Tiempo} = \text{Esfuerzo} / \text{Recursos}$ asume, de forma implícita, que el trabajo de software es perfectamente divisible y paralelizable, como si fuera una tarea mecánica (p. ej. cavar una zanja: el doble de personas hace el trabajo en la mitad de tiempo). El desarrollo de software no cumple esa condición porque el esfuerzo no es una masa homogénea: incluye conocimiento tácito, dependencias secuenciales entre tareas y coordinación entre personas.

Supongamos, de forma simplificada, que el esfuerzo restante del proyecto es de 200 personas-día (5 desarrolladores $\times$ 40 días hábiles restantes). La ecuación naïve predice:

$\text{Tiempo} = 200 / 10 = 20$ días (la mitad del plazo original)

Pero esos 10 recursos no son intercambiables ni están disponibles desde el día 1 con la misma productividad. La ecuación correcta debería incorporar un término de sobrecarga que crece más rápido que el número de personas agregadas:

$\text{Tiempo} = \text{Esfuerzo} / (\text{Recursos} - \text{Sobrecarga (comunicación, onboarding, integración)})$

Es decir, la productividad efectiva de los 10 desarrolladores es menor que el doble de la de los 5 originales — en las primeras semanas puede incluso ser menor que la del equipo original, porque parte del esfuerzo de los seniors se desvía a entrenar, no a producir. Esta es exactamente la Ley de Brooks (1975):

"agregar recursos humanos a un proyecto de software retrasado lo hace más tarde aún".

a) Tres factores de sobrecarga al duplicar el equipo

Tres mecanismos explican matemática y conceptualmente por qué la productividad no escala linealmente con el número de personas:

Factor	Qué ocurre al duplicar el equipo	Efecto sobre el cronograma
a) Comunicación	Los canales de coordinación crecen de forma combinatoria $(n(n-1)/2)$: de 10 a 45 canales posibles, un incremento de $4.5\times$ frente a un equipo apenas $2\times$ más grande.	Más tiempo en reuniones de sincronización, más ambigüedad en decisiones y más superficie de malentendidos; el tiempo disponible para codificar se reduce justo cuando más se necesita.
b) Onboarding / curva de aprendizaje	Los 5 nuevos programadores no conocen el dominio, la arquitectura ni las convenciones del código. Deben ser entrenados por los desarrolladores senior que ya están sobre la fecha límite.	Se produce el efecto que Brooks llama "regresión temporal de productividad": el equipo original reduce su propia velocidad mientras mentorea, generando una caída neta de producción en las semanas más críticas del proyecto.
c) Integración	Más desarrolladores trabajando en paralelo implica más ramas de código, más pull requests simultáneos y más probabilidad de conflictos de merge sobre módulos compartidos.	La integración, las pruebas de regresión y el control de calidad son actividades esencialmente secuenciales (no paralelizables). Duplicar el equipo no acelera ese cuello de botella; solo lo alimenta con más código para revisar.

La sobrecarga de comunicación es la más fácil de cuantificar. La fórmula clásica de canales de comunicación de un equipo de tamaño n es $n(n-1)/2$:

Tamaño del equipo (n)	Fórmula $n(n-1)/2$	Canales de comunicación	Incremento
5 (equipo original)	$5 \times 4 / 2$	10 canales	Línea base
10 (equipo duplicado)	$10 \times 9 / 2$	45 canales	+350% ($4.5\times$)

Lectura del dato: el equipo pasó de $2\times$ en tamaño a $4.5\times$ en complejidad de coordinación. Esa brecha entre el crecimiento lineal del headcount y el crecimiento cuadrático de la comunicación es, matemáticamente, la razón central por la que $\text{Tiempo} = \text{Esfuerzo} / \text{Recursos}$ sobreestima la ganancia real de productividad.

Nota adicional (Amdahl/paralelización): además de la sobrecarga, hay una porción del trabajo que es intrínsecamente secuencial y no se acelera agregando gente. Esa porción secuencial impone un piso mínimo de duración que ningún número de programadores adicionales puede perforar.

b) Dos alternativas de gestión para la crisis de cronograma

Ambas alternativas evitan la trampa de "agregar gente" y, en su lugar, ajustan otra variable del triángulo de restricciones (alcance, tiempo o presupuesto) de forma explícita y negociada, en vez de forzar una promesa que la física del desarrollo de software no puede cumplir.

Enfoque	Acción concreta	Por qué respeta los principios de la ingeniería de software
Predictivo tradicional	Re-línea base del Gantt: recalcular la duración restante con una nueva estimación PERT/COCOMO honesta, y elevar una solicitud formal de cambio (control integrado de cambios, PMBOK 7) al patrocinador para negociar fecha, alcance o presupuesto. Si se requiere más capacidad, aplicar "crashing" selectivo: 1–2 especialistas senior ya familiarizados con el stack, en tareas verdaderamente paralelizables (no la mitad del equipo en bloque).	No se maquilla el cronograma original con una promesa irreal: se comunica la varianza real (Cono de Incertidumbre) y se decide con datos, no con una orden jerárquica que ignora la Ley de Brooks.
Ágil / Híbrido	Negociar el alcance, no el equipo: usar la velocidad histórica del equipo (Story Points/Sprint) para proyectar con datos reales cuánto backlog cabe en los 2 meses restantes, priorizar con MoSCoW/valor de negocio, y entregar un MVP con lo "Must have", moviendo lo demás a un release posterior.	Se ajusta el vértice de alcance del triángulo de restricciones en vez de violar la física del desarrollo de software agregando gente a último momento; se sostiene el flujo del equipo (Kanban, límites WIP) en lugar de romperlo con onboarding forzado.

La orden del director trata el esfuerzo de software como si fuera perfectamente fungible y paralelizable. En realidad, cada persona agregada añade tanto capacidad como fricción, y en un proyecto ya la fricción domina en el corto plazo. La respuesta correcta de un gerente de proyectos moderno no es duplicar el equipo, sino recalcular honestamente la estimación con los datos disponibles (predictivo) o renegociar el alcance con el negocio (ágil/híbrido), y comunicar esa realidad al director antes de comprometer una fecha que la evidencia ya contradice.